Shoeb Sheikh CS23074

Lab 8 Extra Practical: To perform Linear Discriminant Analysis (LDA) and Principal Component Analysis (PCA) separately on a dataset, apply Logistic Regression on the transformed features, and compare the performance of LDA-based and PCA-based models.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
path = "/content/drive/MyDrive/ML_Classes/Breast_Cancer_dataset.csv"
df = pd.read_csv(path)
df.head()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	co points
0	842302	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.
1	842517	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.
2	84300903	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.
3	84348301	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.
4	84358402	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.

5 rows × 32 columns

```
data = load_breast_cancer()
X = data.data
y = data.target
```

```
df.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mea
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.00000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.08879
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.07972
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.00000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.02956
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.06154
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.13070
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.42680

8 rows × 31 columns

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

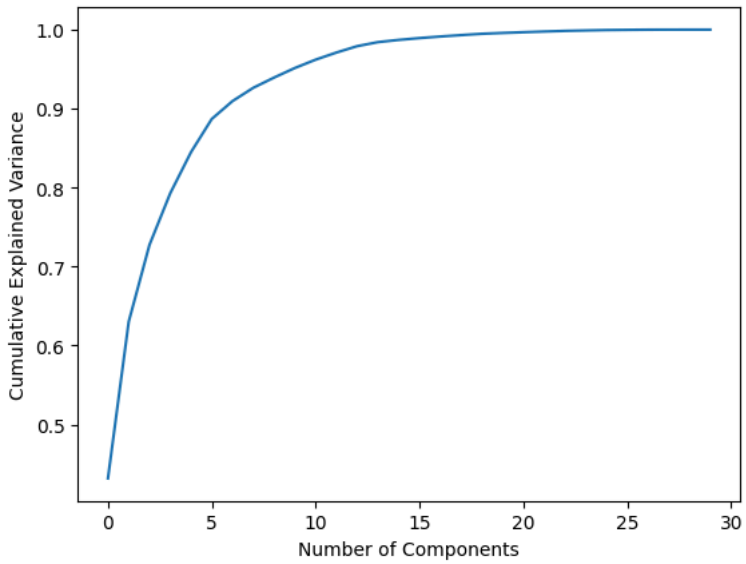
```
df.shape
```

```
(569, 32)
```

```
pca = PCA()
pca.fit(X_train)
```

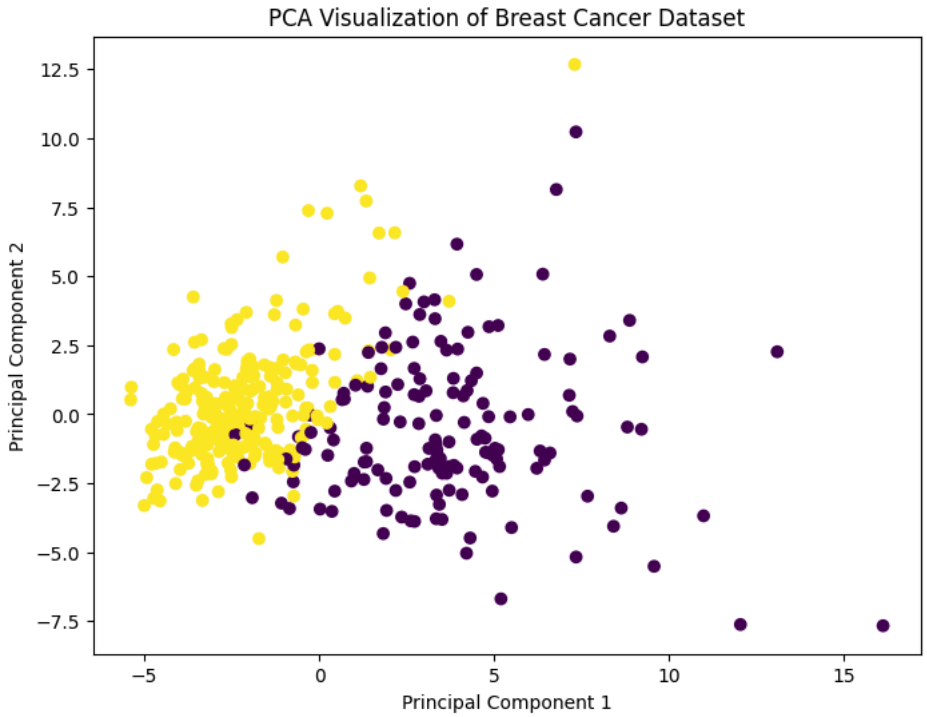
```
PCA
PCA()
```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.show()
```




```
pca = PCA(n_components=2)
pca.fit(X_train)
X_pca = pca.transform(X_train)

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_train)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Visualization of Breast Cancer Dataset')
plt.show()
```




```
lda = LDA(n_components=1)
X_train_lda = lda.fit_transform(X_train, y_train)
X_test_lda = lda.transform(X_test)
```

```
lr_lda = LogisticRegression()
lr_lda.fit(X_train_lda, y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

```
X_train
```

```
array([[-0.12348985, -0.29680142, -0.17050713, ..., -0.84082156,
        -0.8563616 , -0.76574773],
       [-0.22826757, -0.65795149, -0.25377521, ..., -0.37706655,
        -1.3415819 , -0.41480748],
       [ 0.14553402, -1.23056444,  0.24583328, ..., -0.04762652,
        -0.08997059,  0.4882635 ],
       ...,
       [ 0.03226081, -0.55578404, -0.08064356, ..., -1.26179013,
        -0.6828391 , -1.27672587],
       [-0.05552593,  0.10949242, -0.04684166, ...,  1.07924018,
         0.4755842 ,  1.25530227],
       [-0.56525537,  0.32333128, -0.619825  , ..., -0.61952313,
        -0.30366032, -0.84348042]])
```

```
y_train
```

```
array([1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1,
       1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0,
       1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1,
       0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1,
       0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1,
       0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1,
       0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1,
       1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0,
       1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1,
       1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0,
       0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       0, 1])
```

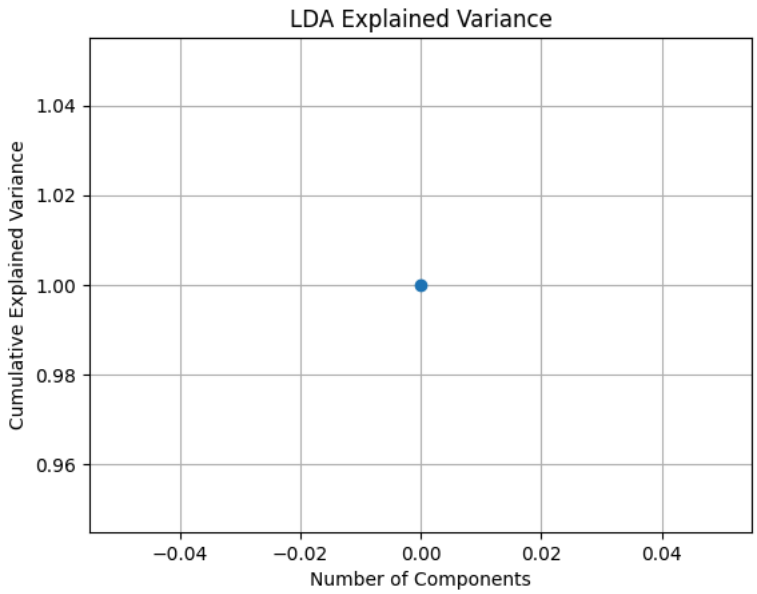
```
y_pred_lda = lr_lda.predict(X_test_lda)
acc_lda = accuracy_score(y_test, y_pred_lda)
```

```
print("Accuracy using LDA + Logistic Regression:", acc_lda)
```

```
Accuracy using LDA + Logistic Regression: 0.9590643274853801
```

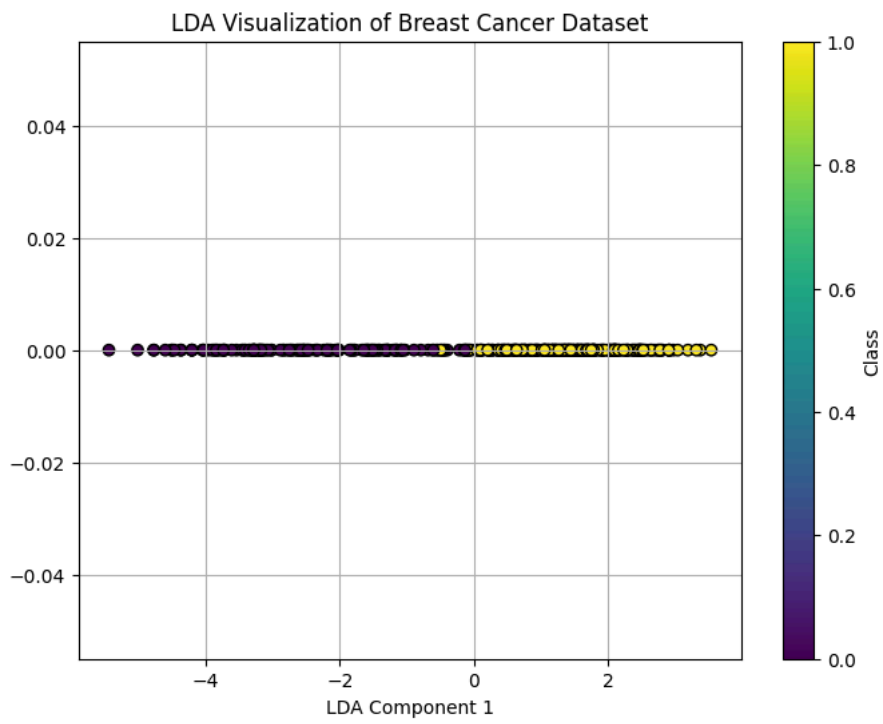
```
lda_cumulative = LDA()
X_train_lda_full = lda_cumulative.fit_transform(X_train, y_train)

plt.figure()
plt.plot(np.cumsum(lda_cumulative.explained_variance_ratio_), marker='o')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('LDA Explained Variance')
plt.grid()
plt.show()
```




```
lda = LDA(n_components=1)
X_lda = lda.fit_transform(X_train, y_train)

plt.figure(figsize=(8, 6))
plt.scatter(X_lda[:, 0], [0]*len(X_lda), c=y_train, cmap='viridis', edgecolor='k')
plt.xlabel('LDA Component 1')
plt.title('LDA Visualization of Breast Cancer Dataset')
plt.colorbar(label='Class')
plt.grid()
plt.show()
```



```
X_train_pca = X_pca
X_test_pca = pca.transform(X_test)

lr_pca = LogisticRegression()
lr_pca.fit(X_train_pca, y_train)

y_pred_pca = lr_pca.predict(X_test_pca)
acc_pca = accuracy_score(y_test, y_pred_pca)

print("\nComparison:")
if acc_pca > acc_lda:
    print("PCA model performs better.")
elif acc_lda > acc_pca:
    print("LDA model performs better.")
```